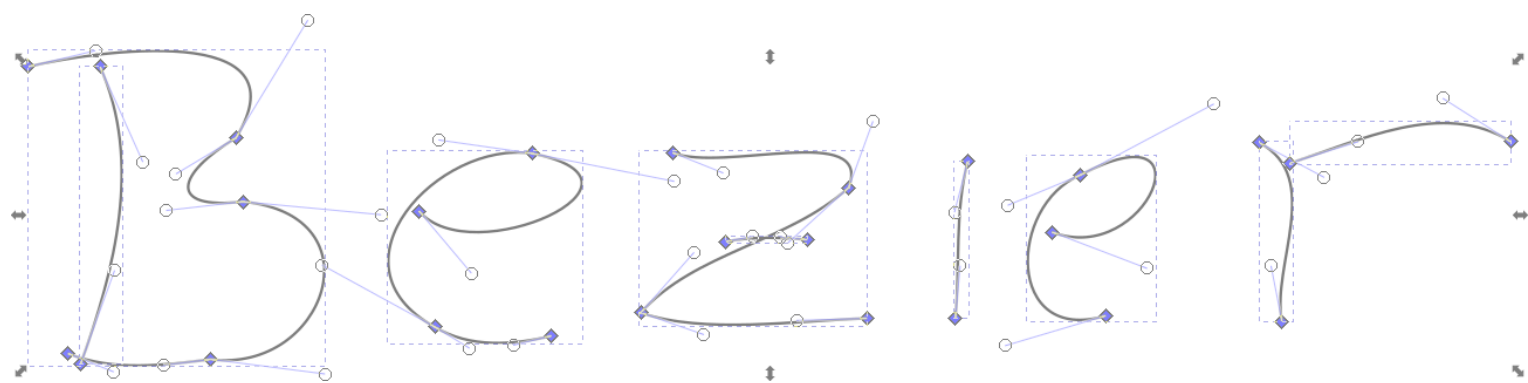


Curvas y Superficies

Parte 1: Curvas

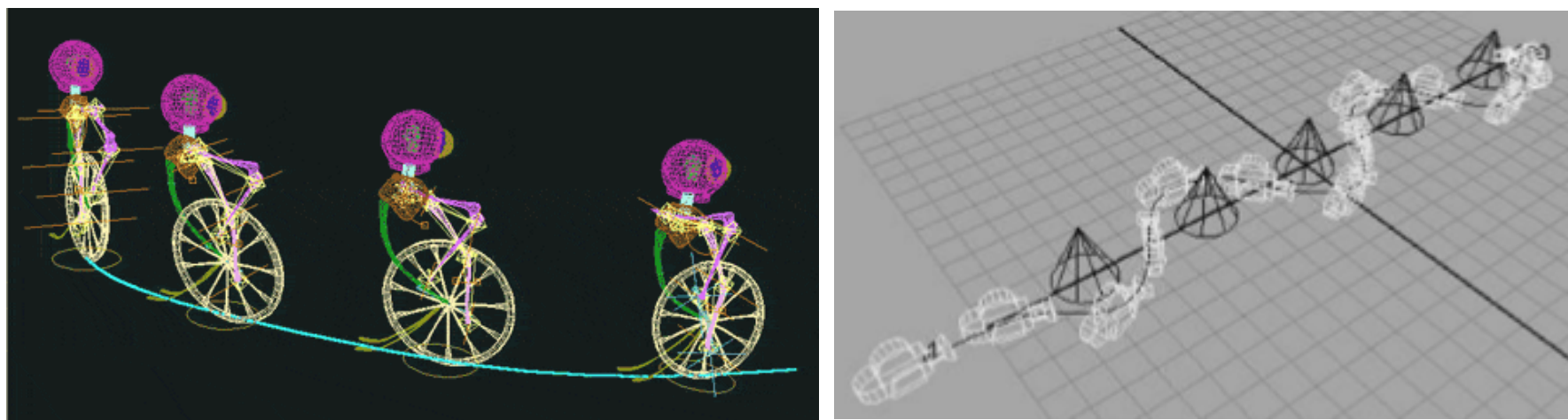


USO DE CURVAS EN CG

- Modelar objetos "suaves" y/u orgánicos



- Definir trayectorias para animar



Estos son los dos usos más obvios:

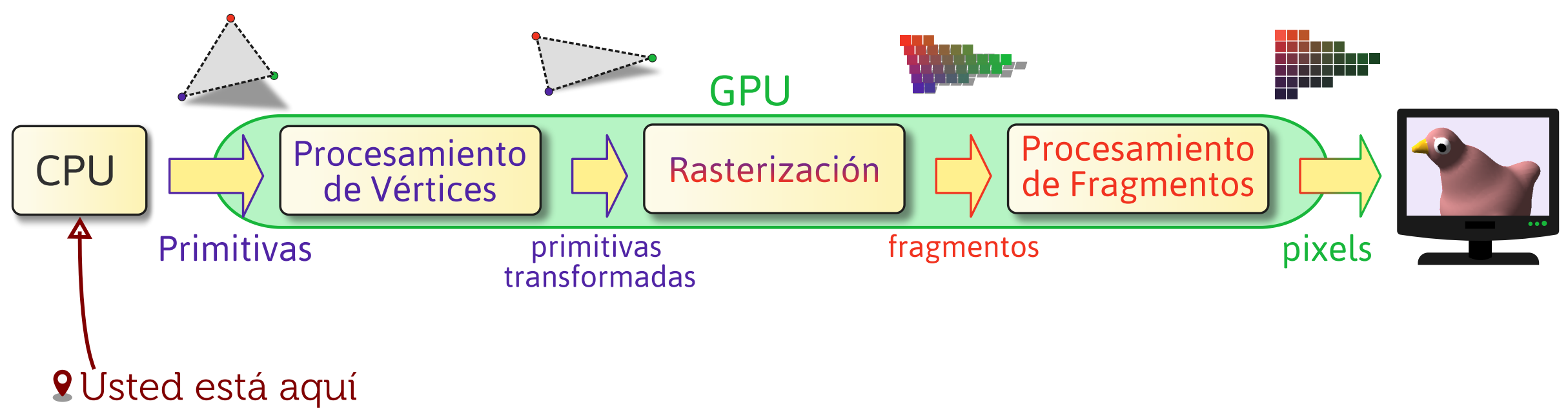
- Modelar "formas" suaves/curvas. Cada caracter de un tipografía, por ej, está modelado con las técnicas para definir curvas que estamos por aprender. Casi todos los casos en donde podemos definir y manipular una curva en un software diseño 2D o similar (Corel Draw, Inkscape, Illustrator, PowerPoint, Word, etc) modela las curvas de esta misma forma.

- Otro caso de uso simple de ver se da cuando queremos predefinir un camino para una animación (el camino que hace un objeto, o el camino por donde se mueve la cámara, etc).

- No son estos dos los únicos casos de uso para la teoría de curvas, solo los más obvios y habituales. Por ejemplo, también se podrá mediante esta teoría entender mejor cómo se pueden usar estas curvas para interpolar (reconstruir un camino o una geometría a partir de una "muestra"),

Vamos a estudiar a fondo ese tipo de curvas usadas habitualmente en 2D, y luego, sobre esa base teórica, veremos (aunque ya sin tanta profundidad) qué otras formas similares de definir curvas se utilizan en casos particulares. Además, esta base teórica para definir curvas luego podrá reutilizarse (la próxima clase) para definir superficies en 3D con fórmulas y propiedades muy similares.

USTED ESTÁ AQUÍ



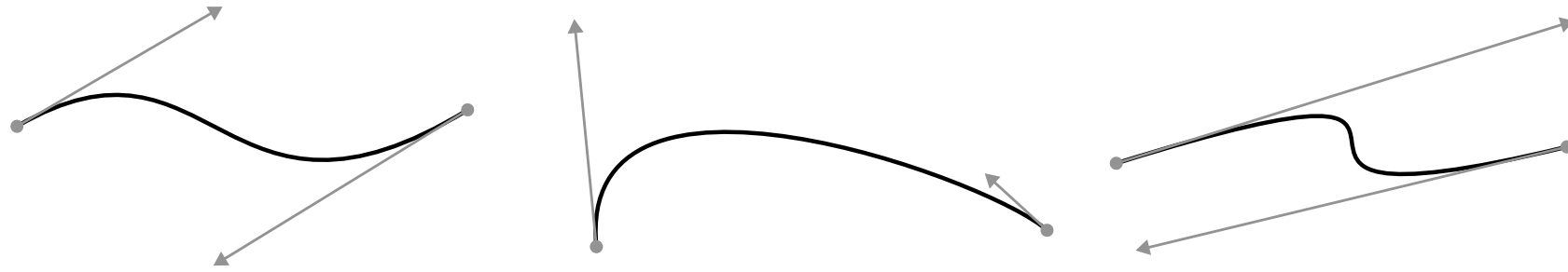
Casi nunca una lista de primitivas es la forma original en la que se definió un objeto. Ya vimos que para el pipeline resulta conveniente y eficiente describir a los objetos mediante primitivas. Pero no es conveniente "diseñar/modelar" a los objetos primitiva a primitiva. En la aplicación es común trabajar con entidades o formas más complejas de representar curvas y superficies, que luego deberán ser discretizadas para ingresar al pipeline.

Por esto, el trabajo con curvas y superficies casi nunca estará dentro del pipeline, sino del lado de la aplicación. Y aún cuando no sea nuestra aplicación la que deba modelar los objetos (en los tps, por ej, los cargamos desde los archivos obj ya "discretizados" por otra aplicación como Blender), habrá muchas oportunidades de utilizar curvas y superficies auxiliares para otras cosas (por ejemplo, para definir una trayectoria sobre la cual luego animar el movimiento de un objeto o de la cámara).

DEFINICIÓN Y REPRESENTACIÓN

Aspectos importantes a tener en cuenta:

- ¿Cómo define el usuario la forma de la curva?



- Qué información se guarda y qué información se calcula
- Cómo se rasteriza o cómo se encuentran los puntos de la curva

Estos son aspectos a tener en cuenta a la hora de pensar o elegir la forma de "representar" un curva en nuestro programa.

- Si la curva será dato de entrada (o nuestro programa permitirá definirla/editarla), es importante que sea fácil de definir para el diseñador que deba generar esos datos. Idealmente, los "parámetros" a modificar para definir la curva deberán ser pocos y tener una clara interpretación geométrica, para que el usuario pueda desarrollar una intuición de cómo funcionan y aprenda a definir las curvas aún cuando no comprenda la matemática que hay detrás.

- Para nuestra aplicación, es deseable que sea eficiente almacenar y manipular la curva. Aquí debemos pensar en qué información es necesario guardar para definirla, y los algoritmos asociados para manipularla.

- Dado que enfocamos gran parte de la materia en el pipeline de renderizado, es deseable que la curva sea fácil de renderizar. Es decir, que se puedan aplicar algoritmos eficientes para hacerlo, ya sea poder aplicar algoritmos específicos para curvas (como DDA), o poder discretizarla para renderizarla mediante primitivas (en la línea del algoritmo de subdivisión).

FORMAS DE DEFINIR UNA CURVA

- Ecuaciones explícitas: $y = mx+b$
 - simple, pero poco general (recta vertical? círculo?)
- Ecuaciones implícitas: $r^2 = (x-x_c)^2 + (y-y_c)^2$
 - útiles para casos particulares (ej: círculo)
 - args. con sentido, orientables, difíciles de graficar
- **Ecuaciones paramétricas:** $p(t) = (1-t)*p_0 + t*p_1$
 - generales y "fáciles" de graficar
- Algoritmos (procedurales): fractales
 - esta idea será mucho más útil para superficies

Explícitas: es un mapeo directo entre una coordenada y las otras ($y=f(x)$). Es la forma con la que generalmente aprendemos funciones en la secundaria, pero no es suficientemente general para computación gráfica pensar en una relación así entre x e y. En la forma $y=f(x)$ no podríamos definir una recta vertical (hay un solo valor de x, muchos de y), o un círculo (hay 2 x por cada y, 2 y por cada x, no es función).

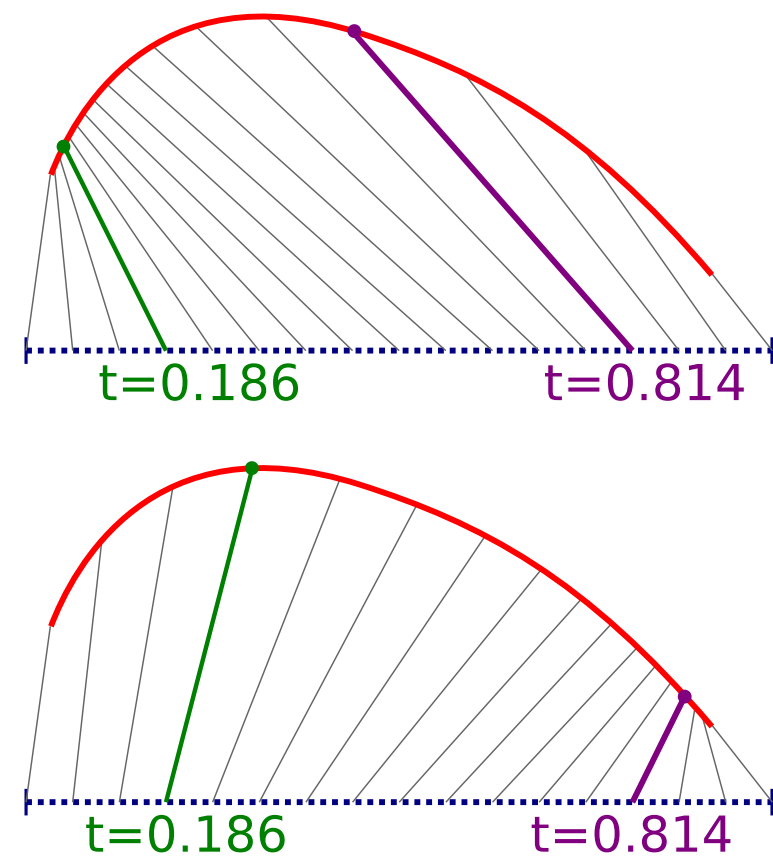
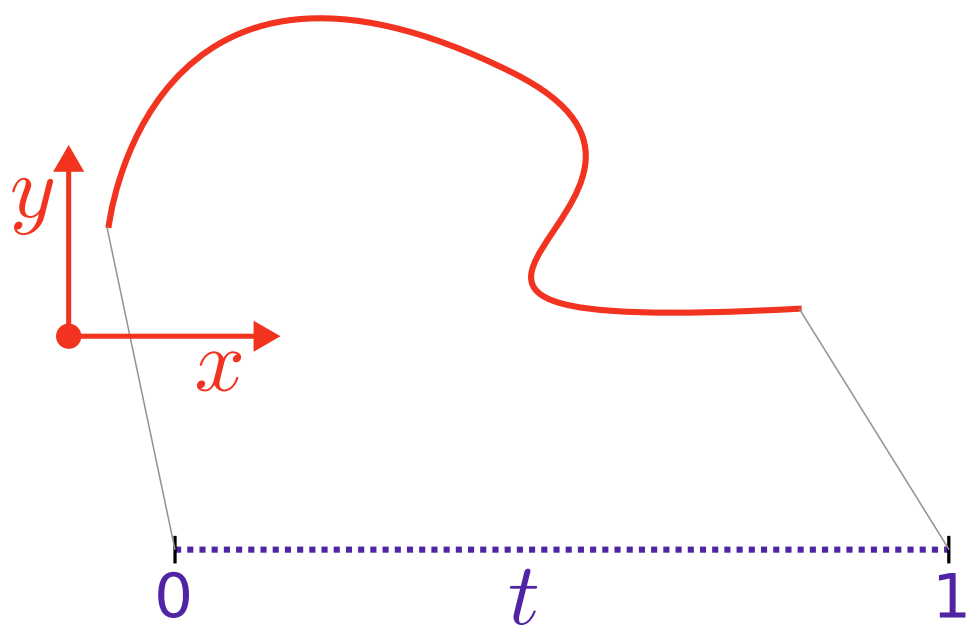
Implícitas: en casos particulares como el ejemplo del círculo son muy útiles para ciertos análisis: es fácil verificar si un punto pertenece, si no pertenece el signo me suele indicar de qué lado estoy (a esto me refiero con "orientables"), los parámetros de la función (dada una forma particular) tienen sentido geométrico (en el círculo, r es el radio, x_c, y_c es el centro), pero son difíciles de graficar, al menos con algoritmos genéricos. No puedo obtener los puntos "probando" pares x,y (que además son infinitos). Se pueden plantear algoritmos específicos y muy eficientes para algunas de ellas (ej: Bresenham de circunferencias), pero siempre son casos particulares.

Las paramétricas definidas así de forma tan general ($x=f(t)$, $y=g(t)$) pueden representar cualquier curva, ya que las funciones f y g podrían tomar cualquier forma. En la práctica restringiremos la forma a cierto tipo de funciones, y ya no representarán absolutamente cualquier cosa, pero sí serán lo suficientemente generales.

Finalmente, se puede pensar en un algoritmo (en lugar de una sola ecuación) que vaya definiendo iterativamente la forma de la curva. Esta idea está aquí por completitud, pero usualmente no se utiliza para curvas, aunque sí veremos más adelante que será muy útil para el caso de superficies.

CURVAS PARAMÉTRICAS

$$\begin{aligned} x &= f(t) \\ y &= g(t) \\ z &= h(t) \end{aligned} \Rightarrow \mathbf{x} = \mathbf{F}(t) = \begin{bmatrix} f(t) \\ g(t) \\ h(t) \end{bmatrix} \quad \text{con } t \in [t_0, t_1]$$



Una curva paramétrica es como una generalización de la explícita: en lugar de definir y en función de x , o x en función de y , se definen ambas en función de un 3er parámetro arbitrario que no necesariamente coincide con ninguna coordenada. A ese parámetro se lo suele denominar t o u .

Nota: en el resto de la presentación se mezcla el uso de t y u en diferentes ejemplos, es trabajo pendiente unificar la notación.

t es un real que puede variar en cualquier intervalo real, pero es común normalizarlo de forma que t varíe entre 0 y 1. La imagen de la izquierda muestra el "mapeo" de cada punto del intervalo real $[0;1]$ a un punto del plano xy . El punto resultante de la curva puede ser de cualquier dimensión (2D en estas figuras), pero la curva en sí es 1D (solo puede "moverse" t).

t puede verse como el tiempo. Si la función $\mathbf{F}(t)$ "dibuja" la curva entre los "momentos" t_0 y t_1 , entonces dado un t , calcular la función es responder a la pregunta ¿dónde estaría el "lápiz" en el tiempo t ?

Una vez "dibujada" la curva de forma continua, es imposible saber cuál era la parametrización. Para ese resultado final la parametrización es "invisible". Pero si el objetivo no es solo el "dibujo" final (por ejemplo, si ese dibujo es en realidad la trayectoria de un objeto en una animación), entonces el tiempo sí importa y se nota.

REPARAMETRIZACIÓN

- **Normalización** del dominio:

$$\mathbf{F}(t), \quad t \in [t_0, t_1] \rightarrow \hat{\mathbf{F}}(u), \quad u \in [0, 1] \Rightarrow \hat{\mathbf{F}}(u) = \mathbf{F}(t(u))$$
$$t \in [t_0, t_1] \Rightarrow t = (1 - u)t_0 + ut_1 \Rightarrow t(u) = t_0 + u(t_1 - t_0)$$

- Por **longitud de arco** (velocidad constante):

$$u = \int_0^t \sqrt{[\mathbf{F}'(v)]^2} dv$$

⊗ *En general no será posible resolver esta integral analíticamente.*

"Reparametrizar" una curva es obtener una nueva ecuación que genere el mismo conjunto de puntos (el mismo "dibujo") pero con una parametrización (una velocidad) diferente.

Una reparametrización "afín" es muy simple de hacer, y la usaremos más de una vez. El caso de uso más obvio es al "renormalizar" un parámetro t que estaba en un intervalo diferente al $[0;1]$. Podemos encontrar t como una combinación afín de t_0 y t_1 , y entonces al reemplazar t por esta combinación en la función de la curva, nos queda una nueva función donde el parámetro es ahora el peso de la combinación afín, que varía entre 0 y 1.

Una reparametrización que sería muy útil/interesante lograr es la llamada "por longitud de arco". Esto es, una parametrización en la que el "lápiz" avance a velocidad constante sobre la curva. Matemáticamente es simple de plantear, pero casi siempre imposible de resolver de forma analítica. Por esto, no usaremos nunca esta integral, sino que en la práctica si es necesario se aproximará numéricamente. Es un ejercicio interesante (y una potencial pregunta de parcial) pensar cómo sería un algoritmo para realizar esta aproximación.

CURVAS PARAMÉTRICAS: FUNCIONES BASE

Hay que definir la "forma" de la función $\mathbf{F}(t)$:

- Que se pueda definir y editar "fácilmente":
 - Pocos parámetros
 - Parámetros con significado geométrico
 - Eficiencia de los cálculos
- Más usadas:
 - **Polinomios/Series de Potencias**
 - Trigonométricas

Todavía no habíamos especificado qué tipo de funciones serían las que componen $\mathbf{F}(t)$. Idealmente deberían ser funciones que se puedan evaluar eficientemente, que se puedan definir mediante pocos parámetros, y que esos parámetros tengan algún sentido geométrico para que sean fáciles de definir, aún por un usuario inexperto.

En computación gráfica las funciones que se usan para todo son polinomios, y en general de grado bajo (2do, 3ro, 4to). Veremos a continuación cómo definirlos para que sus parámetros (los coeficientes de cada término) tengan un significado geométrico intuitivo.

En otros campos son muy utilizadas funciones bases trigonométricas, y esto tiene más que ver con descomponer la función en una suma de funciones de diferente frecuencia (véase Fourier), algo útil para cierto tipo de análisis. Verán esto en otra materia (señales, imágenes), pero en general no es el caso de computación gráfica.

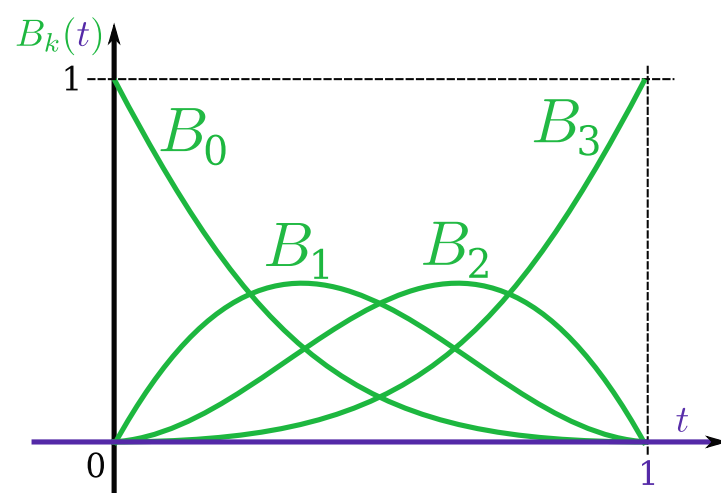
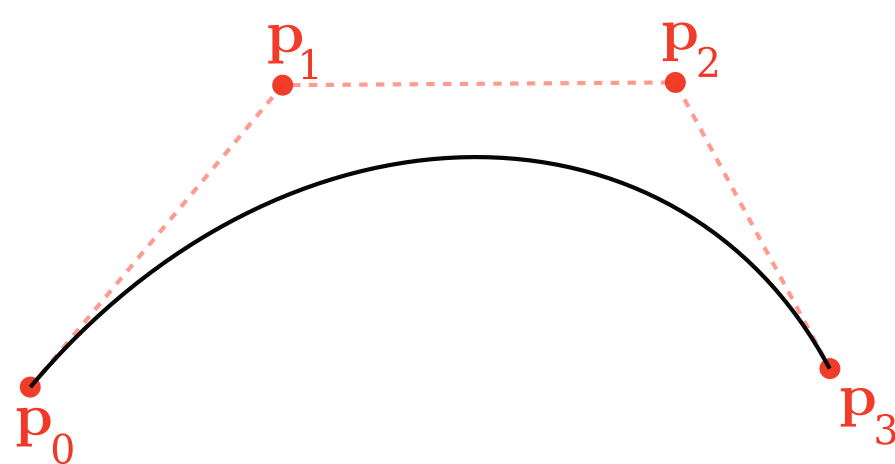
SERIES DE POTENCIAS Y FUNCIONES BASE

Si las funciones en $\mathbf{F}(t)$ son polinomios de grado N :

$$\mathbf{F}(t) = \sum_{i=0}^N \mathbf{a}_i t^i \quad \Rightarrow \quad \mathbf{F}(t) = \sum_{i=0}^N \mathbf{p}_k B_k(t)$$

Una mejor forma de definir las mismas funciones es mediante:

Puntos de control y **Funciones Mezcladoras**

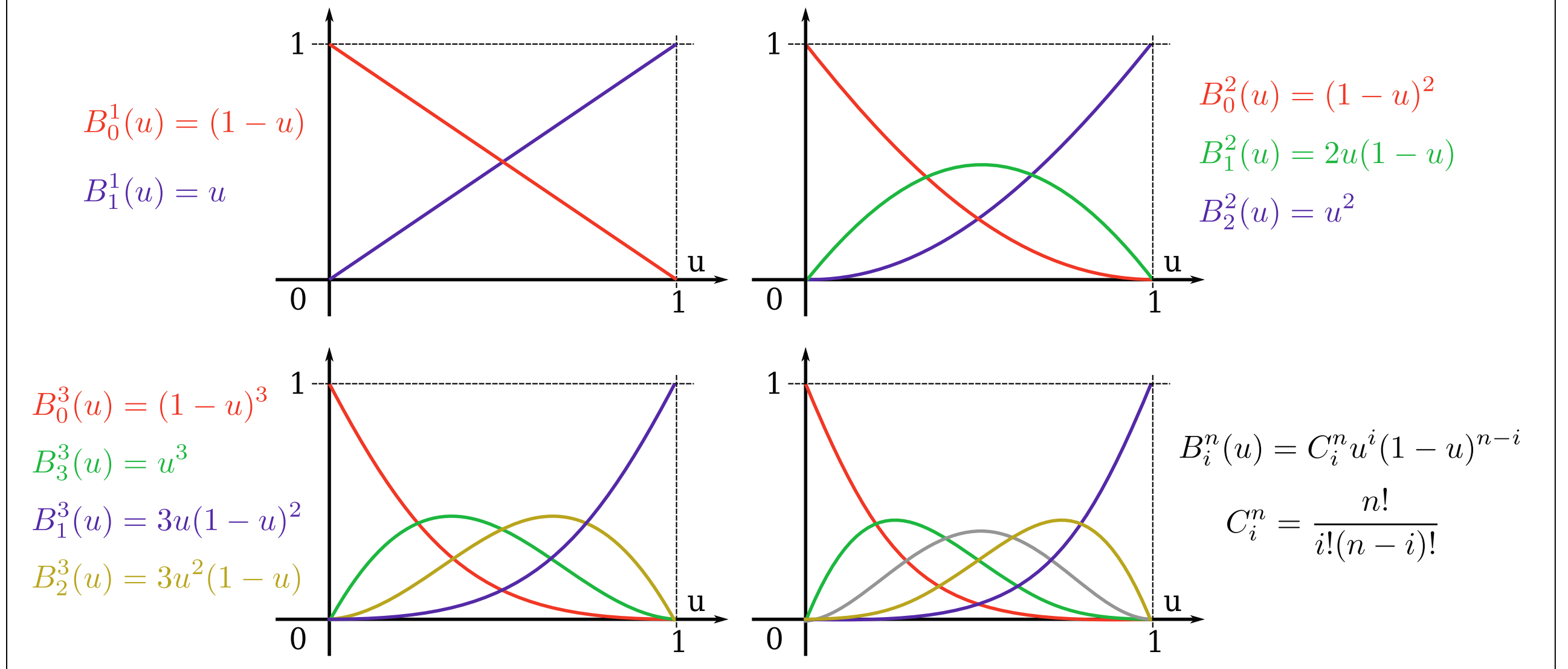


La forma más natural de definir un polinomio es mediante los coeficientes de cada término. Si es de grado N , tenemos $N+1$ coeficientes. Para la curva, como en realidad hay un polinomio para cada coordenada (una función para x , otra para y , otra para z), entonces hay $N+1$ vectores de coeficientes (se dice grado N , orden $N+1$). En la fórmula de la izquierda son los vectores $\mathbf{a}_i$. Notar que es como una combinación lineal de monomios, donde $\mathbf{a}_i$ son los pesos, y los monomios t^i las "bases" a combinar. Estos monomios serían como la "base canónica" para armar un polinomio, pero no es la única.

En la fórmula de la derecha, armamos los polinomios combinando linealmente una "base" diferente. Si estas funciones "base" $B_k(t)$ siempre suman 1 (para todo t), entonces se puede ver como que estas funciones van dando "pesos" para combinar afinmente a los puntos $\mathbf{p}_k$. Ahora los "coeficientes" de la combinación lineal tienen un sentido geométrico más claro: son en realidad coordenadas de puntos. Por eso los llamamos "puntos de control" (y al polígono que forman, "polígono de control"). Y a las funciones, "funciones mezcladoras" (blending functions) porque dicen cómo "mezclar" (pesar) esos puntos de control para ir obteniendo los puntos de la curva.

CURVAS DE BÉZIER

Si las "blending functions" son los **Polinomios de Bernstein**
entonces estamos hablando de **Curvas de Bézier**



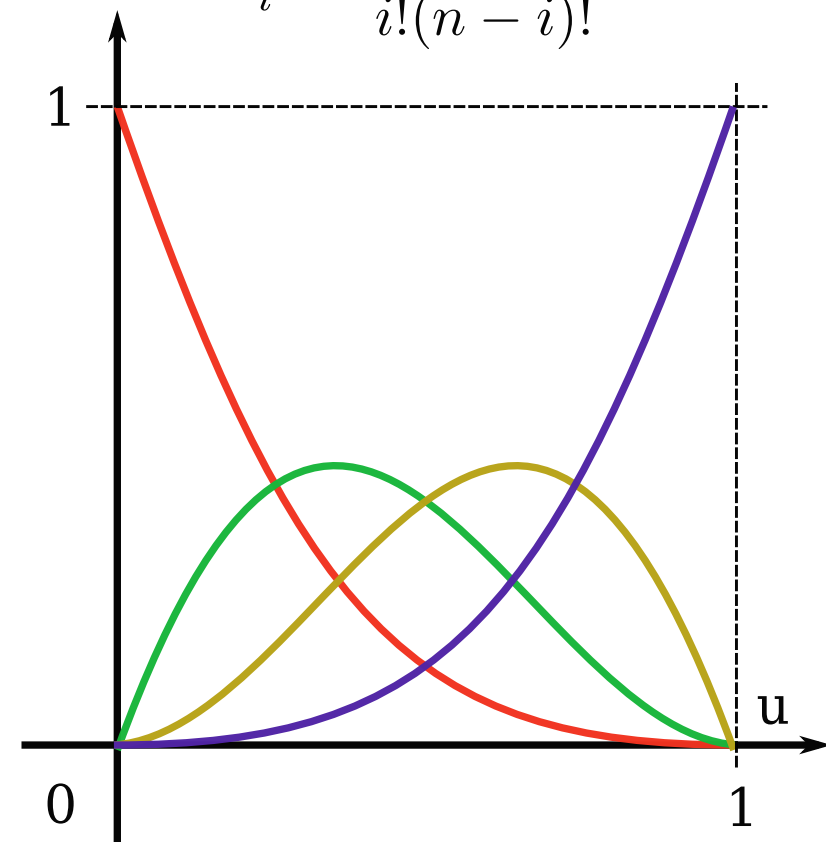
Las Curvas de Bézier son las que vamos a estudiar en mayor detalles. Son curvas polinómicas. Los polinomios que se utilizan como bases (funciones mezcladoras) para definir estas curvas son los "polinomios de Bernstein".

La definición general (polinomio i -ésimo para una curva de grado N) aparece abajo a la derecha; los otros 3 casos muestran casos particulares (bases para curvas de grado 1, 2 y 3, que serán los más habituales). Pero no hace falta recordar ninguna de estas fórmulas, ya que podrán deducirse fácilmente a partir de un algoritmo que veremos más adelante (De Casteljau).

POLINOMIOS DE BERNSTEIN: PROPIEDADES

$$B_i^n(u) = C_i^n u^i (1-u)^{n-i}$$

$$C_i^n = \frac{n!}{i!(n-i)!}$$



- ▶ Interpola puntos extremos
- ▶ Aproxima puntos interiores
- ▶ Control global
- ▶ Combinaciones afines y convexas
- ▶ Unicidad
- ▶ Simetría respecto a $u=0.5$
- ▶ Suavidad

Los polinomios de Bernstein, que son la base para las curvas de Bézier, tienen varias propiedades útiles. No las vamos a demostrar, pero sí veremos que las gráficas de estas funciones permiten "deducir" (intuir en realidad) visualmente muchas de ellas. Y en las siguientes slides le encontraremos sentido práctico a estas propiedades:

- En $t=0$ y $t=1$ ocurre que una de las funciones bases (la 1ra para $t=0$, la última para $t=1$) vale 1 y todas las demás valen 0. Si el peso de un punto de control es 1 y los de los demás 0, entonces la combinación da ese punto. Esto es, la curva "pasa" por ese punto. Para $t=0$, la curva pasa por el 1er punto de control, y para $t=1$, la curva pasa por el último. De aquí que decimos que **"interpola"** (en oposición a "aproxima") **a los puntos extremos**.

- Dentro de $(0,1)$, para cualquier otro valor de t , no vuelve a ocurrir que un peso sea 1 y los demás 0. Dentro del intervalo (abierto) de 0 a 1, todas las funciones valen "algo" distinto de cero (aunque sea "algo" muy pequeño). Esto tiene 2 implicancias:

- por un lado lo que llamamos **control global**: el movimiento de un punto de control cambia toda la curva (con la única posible excepción de los extremos). Puede que la cambie poco (por ejemplo, cerca de $t=0$ todas las funciones menos la 1ra pesan muy muy poco), pero es un cambio al fin.

- por otro lado, la curva no "interpolará" ningún otro punto de control. Sí les pasará "cerca", ya que cada punto tiene su influencia, y para cada punto hay algún tramo de la curva donde ese influye (pesa) más que el resto. Pero entonces la curva solo **aproxima a los puntos de control interiores** (no extremos).

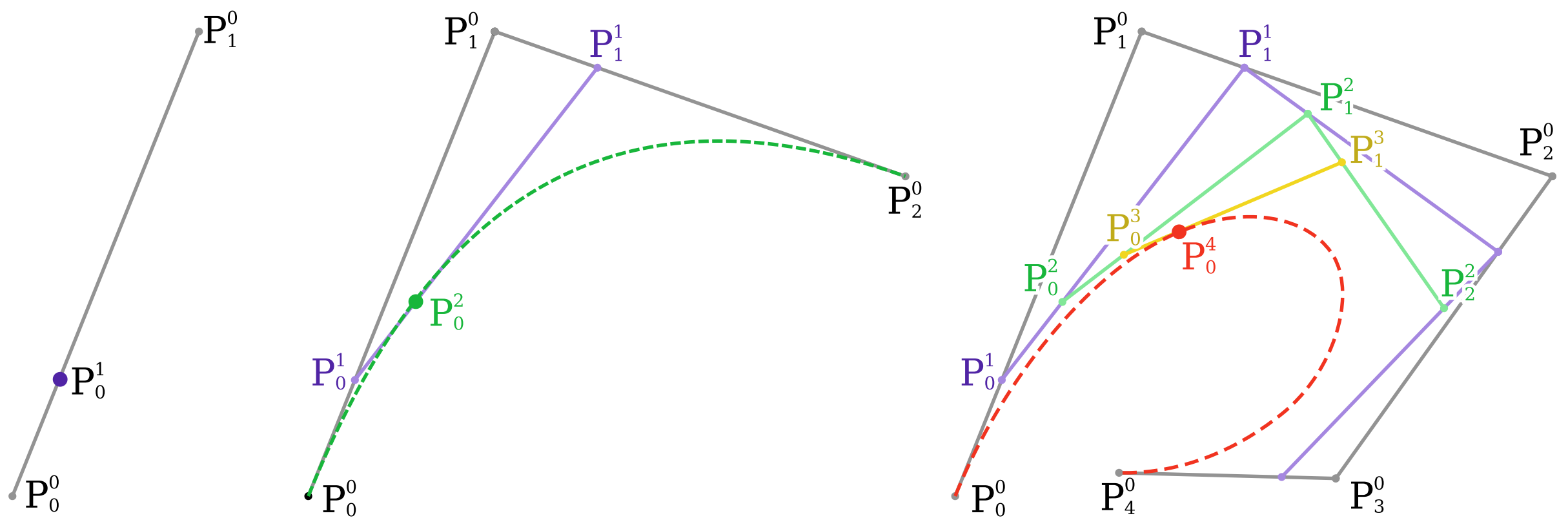
- Se ve en la gráfica que todos los pesos están siempre entre 0 y 1, lo cual implica que si **la combinación es afín** (cosa que ya dijimos, pero "todavía" no están obvios que sumen 1) **también es convexa**. Esto implica que la curva **nunca saldrá del convex-hull** que definen los puntos de control.

- La **unicidad** refiere a que dada la secuencia de pts de control, hay una sola curva posible. No se puede obtener una cierta curva de dos formas diferentes usando las bases de Bézier y manteniendo el grado (sí se puede a veces si las dos "formas" son de distinto grado). La única posible excepción es cuando invertimos el orden de los puntos: la curva no es la misma (porque ahora avanza en sentido inverso), pero su imagen (el conjunto de puntos de la misma, como conjunto, sin importar el orden) sí. Esto es gracias a que el conjunto de funciones base es **simétrico respecto a $t=0.5$** .

- Finalmente, como las bases son polinomios, y los polinomios son infinitamente derivables, tenemos ciertas garantías de **suavidad** de la curva.

ALGORITMO DE CASTELJAU

Algoritmo para (dado un t) obtener un punto de la curva:



¿Qué tiene que ver con los polinomios de Bernstein?

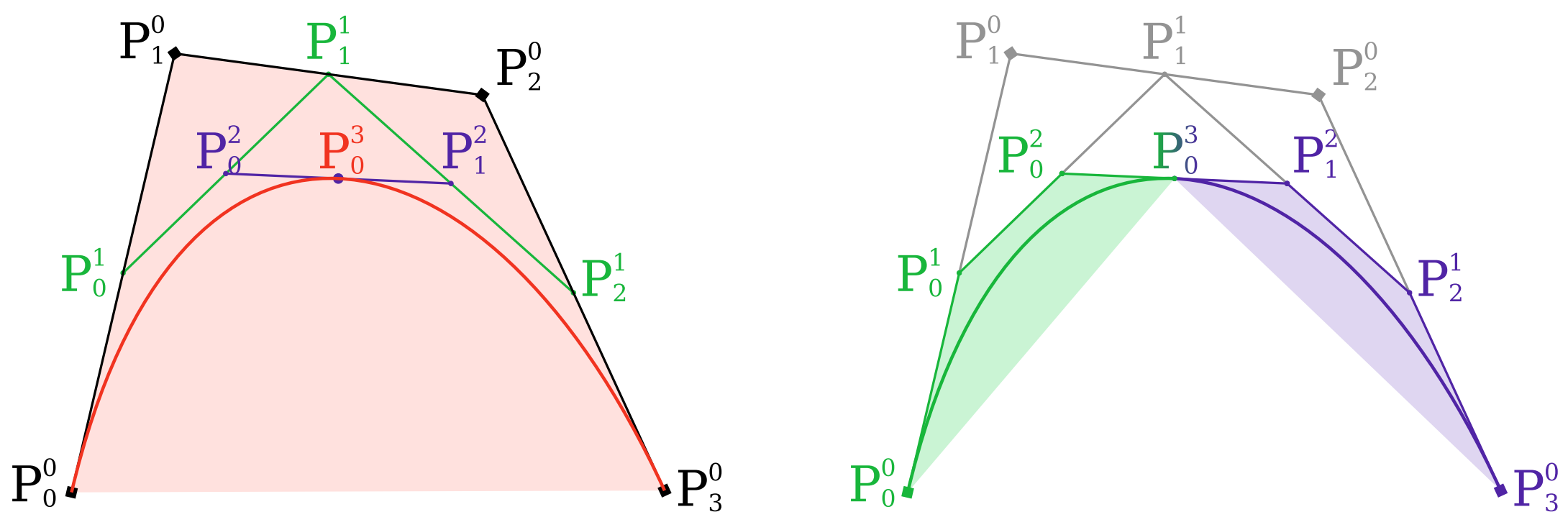
El algoritmo de "De Casteljau" nos permite, dado un valor para el parámetro t , obtener un punto de la curva (y ya veremos que salen algunas cosas útiles más en el proceso). Se basa en hacer muchas interpolaciones afines entre 2 puntos:

1. Primero, en cada segmento del polígono de control, se interpola entre sus dos puntos, siempre con el mismo parámetro t .
2. Como resultado, si el polígono de control original tenía N puntos, ahora tenemos $N-1$ nuevos puntos. Con esto podemos armar un nuevo polígono de control más "corto" (para una curva de un grado menos).
3. Repetimos el proceso en ese nuevo polígono de control (es decir, volvemos al paso 1). En cada iteración tenemos un nuevo polígono, cada vez más corto, hasta que quede un solo punto de control (ese será el punto de la curva para el t dado).

Si planteamos las "recursivas" interpolaciones afines, llegamos a los polinomios de Bernstein. Por esto no hace falta saber la forma general, siempre se puede "reconstruir" cada polinomio base con este proceso. Aunque sería tedioso para grados altos, casi siempre trabajaremos con curvas de grado 2 o 3.

ALGORITMO DE CASTELJAU: SUBDIVISIÓN

Aplicando el algoritmo, además del punto $\mathbf{F}(t)$, obtenemos una subdivisión de la curva (pts de control para dos sub-curvas):



✓ El tamaño del Convex-Hull se reduce cuadráticamente

Una de los "subproductos" del algoritmo de De Casteljau, son los puntos necesarios para representar las dos "mitades" de la curva (no hace falta en realidad que $t=0.5$, aplica para cualquier t).

Se observa en la figura que la curva original (roja) se puede representar como la unión de las curvas verde y azul; y que los puntos de control de estas dos "sub-curvas", son los puntos que va generando De Casteljau a través de las iteraciones al evaluar la original.

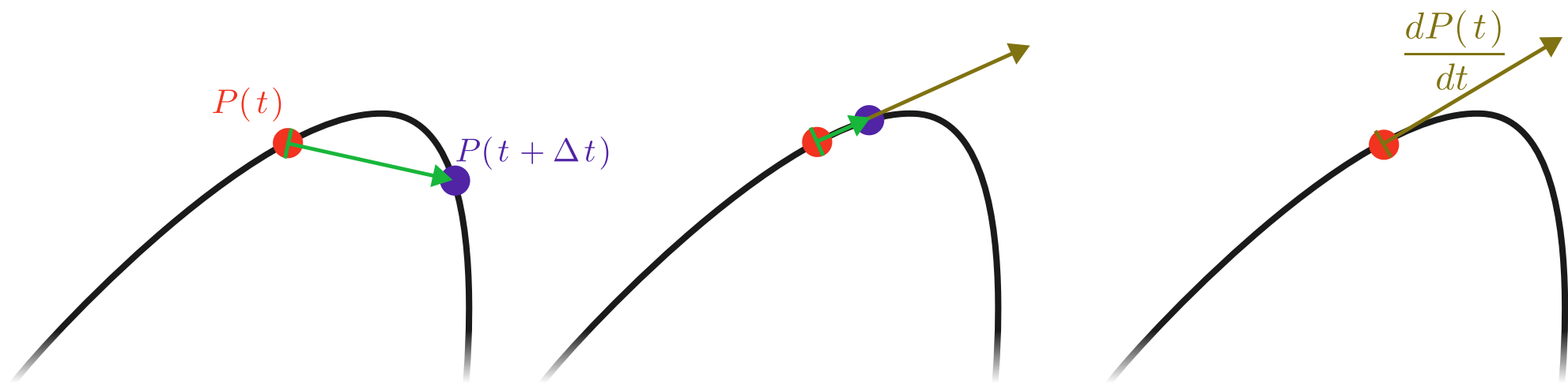
Esto, sumado a la propiedad del convex-hull (que la curva no puede salir del mismo), y a que al subdividirla de esta manera los convex-hulls de las dos nuevas sub-curvas serán "mucho" más chicos que el original, sirve para resolver muchos problemas.

- Si quiero, por ejemplo, saber si una recta corta a la curva, puedo primero ver si corta al convex-hull. Si no lo hace, ya se que no corta a la curva tampoco. Si sí lo hace, tengo que "refinar" mi resultado. Ahí aplico esta subdivisión. Si lo hace, subdivido a la mitad y veo si intersecciona a alguno de los nuevos convex-hulls. Y así recursivamente, hasta descartar la intersección, o hasta quedarme con un convex-hull tan chico como la precisión que necesite.

- Al decir que los convex-hulls se reducen "mucho", estamos diciendo que este algoritmo recursivo llegará relativamente rápido (sin requerir tantos niveles de recursión) a la precisión que necesitemos. Se puede demostrar (aunque no lo haremos) que "mucho" en este caso es cuadráticamente.

DERIVADA DE UNA CURVA

$$\frac{dP(t)}{dt} = \lim_{\Delta t \rightarrow 0} \frac{P(t + \Delta t) - P(t)}{\Delta t}$$



🤔 ¿Qué es la derivada de una curva?

Hacemos una pausa en Bézier para volver a algo más general de cualquier curva: ¿Qué es la derivada de una curva?

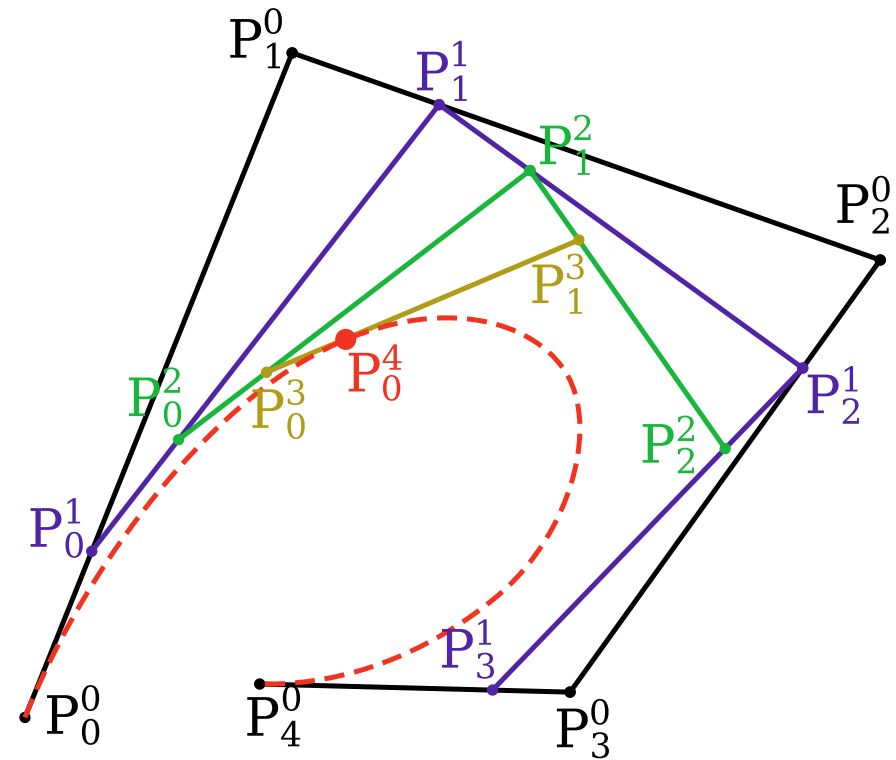
Es un error común escuchar respuestas como "la pendiente de la recta tangente" o ese tipo de cosas que no son más que el producto de haber aprendido derivadas inicialmente con funciones de la forma $y=f(x)$.

Pero en general, la definición de la derivada sale del cociente entre el cambio en la imagen $f(x)$ sobre el cambio en de la variable independiente (x), cuando este último cambio tiende a 0 (es un límite). En este caso, la variable independiente es t , un real; y la imagen $f(t)$ es un punto. Entonces, en ese cociente tenemos arriba una diferencia de puntos (o sea, un vector) y abajo un escalar: el resultado sigue siendo un vector, que puesto sobre el punto de la curva saldrá tangente a la misma, y cuya magnitud tendrá que ver con la velocidad con que el punto avanza sobre la curva a medida que cambia t (o sea que depende de la parametrización!).

DERIVADA DE UNA CURVA DE BÉZIER

El último segmento del algoritmo de DC define la derivada:

$$\frac{d}{du} \mathbf{P}_0^n = n(\mathbf{P}_i^{n-1} - \mathbf{P}_0^{n-1})$$



- n es el grado de la curva
- $\mathbf{P}_i^j$ es el i -ésimo punto en la j -ésima iteración de DC

La derivada de una curva de Bézier se puede obtener analíticamente derivando su fórmula (esto es, derivando los polinomios de Bernstein). El ejercicio de hacerlo es simple pero tedioso, y sirve para notar algunas propiedades interesantes en el camino. La más importante/útil es que la derivada puede plantearse como el vector que define el último segmento del algoritmo de De Casteljau (el de la iteración en que el polígono de control es solo un segmento, justo antes de hacer la última combinación afín) multiplicado por el grado de la curva.

Esto nos da una forma fácil de calcularla: así como no necesitamos recordar la forma general de los polinomios de Bernstein si recordamos el algoritmo de De Casteljau, tampoco necesitamos derivar las funciones base. Pero además nos da una forma "gráfica" de encontrarla, con la cual será más simple/intuitivo hacer cosas.

Algo que por ejemplo podemos notar rápidamente, es que si nos paramos en $t=0$, ese último segmento en realidad coincide con el primer tramo del polígono original (habrá muchas interpolaciones:

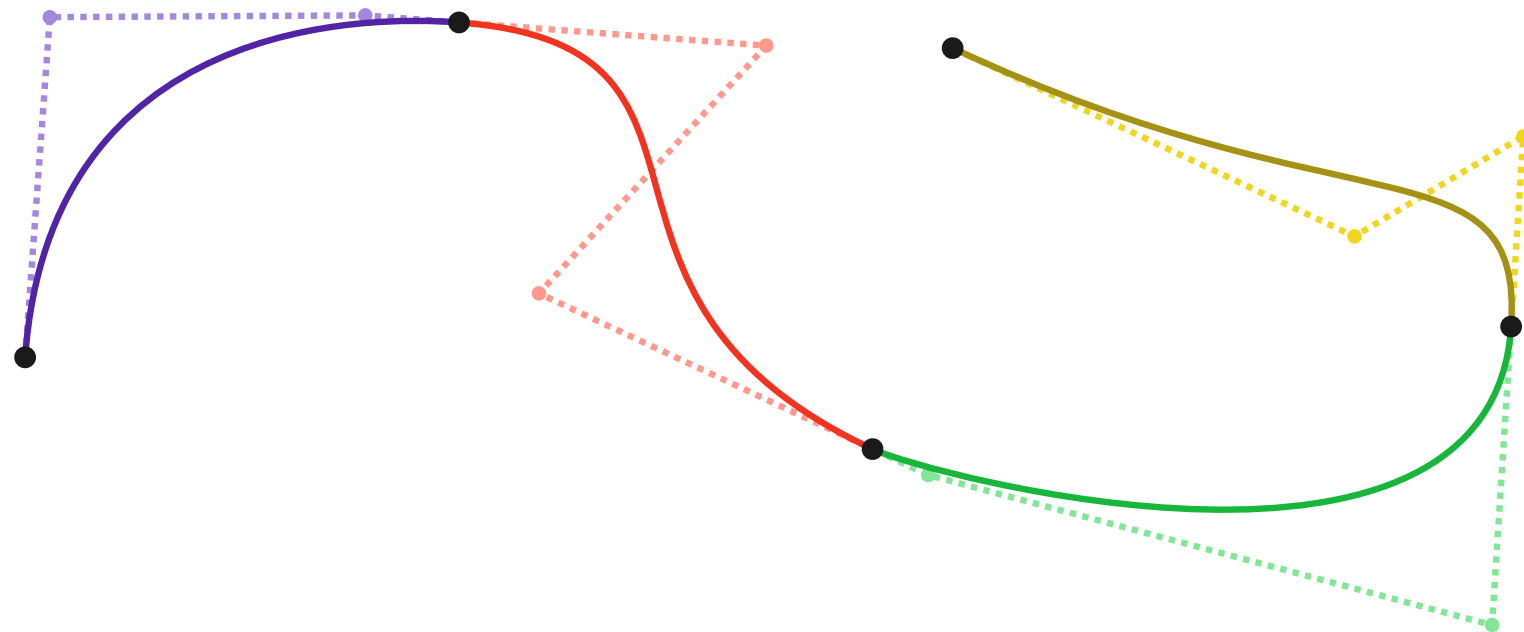
$$P_i^{j+1} = (1-0)*P_i^j + 0*P_i^{j+1}, \text{ o sea } P_i^{j+1} = P_i^j.$$

Entonces, la curva parte del primer punto de control, pero además ahora sabemos que sale tangente al primer segmento del polígono. Aplicando la propiedad de simetría, podemos demostrar/deducir que lo mismo se cumple al final (la curva llega al último punto de control de forma tangente al último segmento del polígono).

Esto será inmediatamente útil (en la slide que sigue), ya que dentro de una curva tenemos ciertas garantías de "suavidad" gracias a las funciones base, pero donde se "conecten" dos curvas deberemos tener cuidado de igualar sus derivadas si queremos que la "conexión" no se note.

SPLINES DE BÉZIER

Piecewise Bézier Curve (Bézier por tramos)



Una **spline** es una curva definida por tramos, unidos de forma de lograr cierto grado de continuidad.

► Control local, bajo grado/orden

🤔 ¿Qué grado usar en c/tramo?

Una curva de un grado bajo no puede describir una forma muy compleja. Pero una curva de grado alto es difícil de definir y manipular (control global). Entonces, para definir formas complejas se usa la idea de spline. Una spline es una unión de dos o más curvas para formar algo que parezca una solo curva más compleja.

Las curvas más utilizadas son las splines de Bézier de grado 3: uniones de curvas de Bézier de grado 3.

Con una spline logramos control local, ya que al editar un punto de control se modifican como mucho dos tramos (2 de las N curvas unidas), y el resto permanece constante. Por esto es más fácil para el usuario manipular una spline de grado bajo antes que una única Bézier de grado alto.

Al unir dos curvas, buscaremos cierto grado de continuidad. Por supuesto que la 2da curva debe empezar donde termina la 1ra para que al menos sea una solo "cosa" continua. Pero además buscaremos hacer coincidir las derivadas para que el punto de unión no sobresalga como un punto anguloso dentro de una curva por lo demás suave.

El grado que se usa habitualmente es 3. Con grado 3, tenemos 4 puntos de control (o 4 grados de libertad para definir la curva). Es intuitivo definir dónde empieza y dónde termina (2 pts de control/2 grados de libertad) y "cómo" (dirección/velocidad) empieza y termina en esos puntos (los 2 pts/grados restantes). Sería más complicado encontrarle alguna interpretación intuitiva a un 5to punto de control. Sería difícil (o imposible) conectar bien los tramos con un grado menos, ya que luego de definir el punto inicial y la derivada en ese punto para conectar una segunda Bézier a la primera, nos quedaría un solo punto libre/grado de libertad para definir qué ocurre al final (podríamos controlar la posición donde termina, pero ya no la derivada).

CONTINUIDAD PARAMÉTRICA VS. GEOMÉTRICA

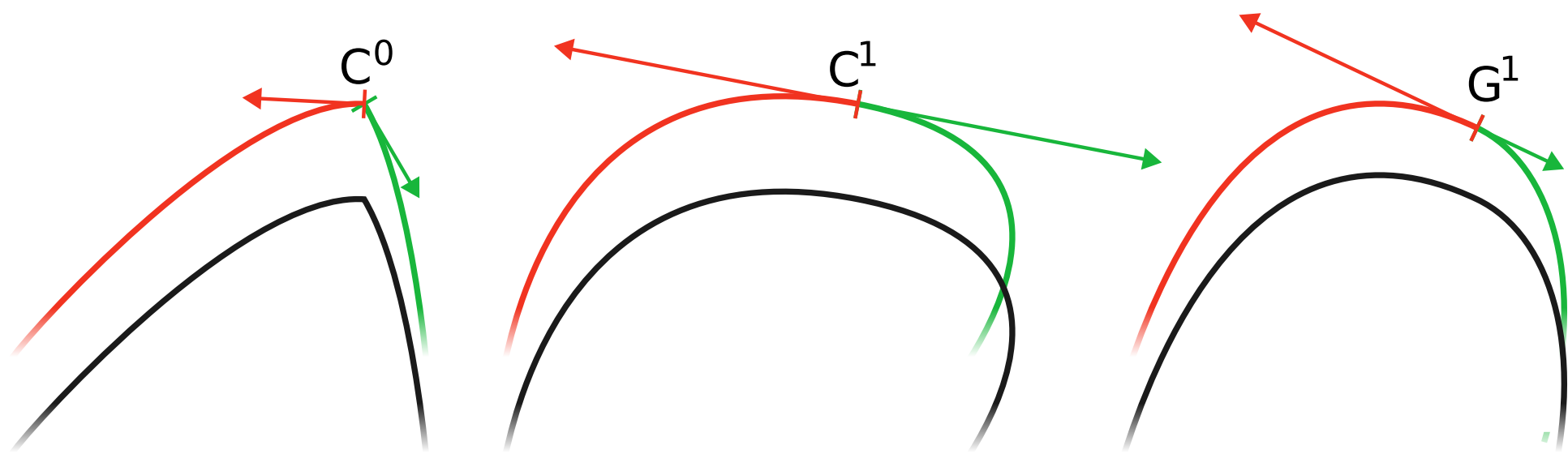
- Continuidad Paramétrica

- Derivadas iguales

- Continuidad Geométrica

- Derivadas Proporcionales ($K > 0$)

✓ *Indepete. de la parametrización*



Al unir dos curvas de Bézier y buscar que la unión sea "suave", se podría pensar inicialmente que se necesita que las derivadas sean iguales: la derivada de la primera curva al final, en $t=1$; y la derivada de la 2da al comienzo, en $t=0$. A esto se le llama **continuidad paramétrica**. Se puede hablar de grados de continuidad paramétrica (por ej C^2 implicaría que las 2das derivadas también coinciden), aunque con Splines de Bézier definidas como hasta ahora, casi siempre nos concentraremos solo en la primera derivada.

Sin embargo, la continuidad paramétrica en la mayoría de los casos es más de lo necesario, aunque puede que en algunos casos especiales no sea suficiente (si las derivadas en el punto en cuestión son el vector nulo, la curva podría generar visualmente un punto anguloso ya que ese vector no implica dirección alguna).

Usualmente, para que la curva se "vea" suave, alcanza solo con que las derivadas sean no nulas y proporcionales: vectores con la misma dirección y sentido, no necesariamente la misma magnitud ($v_1 = kv_2$, la cte de proporcionalidad k debe ser estrictamente positiva). A esto se le llama **continuidad geométrica**, y definido así solo tiene sentido para la primera derivada (G^1).

Pedir continuidad geométrica es como pedir continuidad paramétrica si ambas curvas estuvieran parametrizadas por longitud de arco. O dicho de otro modo, es como pedir la continuidad que se puede pedir independientemente de la parametrización. Sirve cuando nos interesa la "gráfica" de la curva. Pero si la vamos a "recorrer" (por ejemplo, para animar la posición de la cámara), vamos a necesitar continuidad paramétrica (o reparametrizar luego, en cualquier caso si no la cámara cambiaría su "rapidez" de golpe al cambiar de tramo de Bézier).

Dado que en Bézier la derivada en los extremos sale del segmento del polígono de control, unir dos Bézier del mismo grado con continuidad geométrica es alinear los dos "3" pto de control (los dos últimos de una curva y los dos primeros de la otra, siendo el último y el primero iguales, por eso digo 3). Para lograr continuidad paramétrica, además de estar alineados deben estar equidistantes.

B-SPLINES

Base polinómica (Cox/de Boor):

$$B_k^0(t) = \begin{cases} 1 & \text{si } u_k \leq t \leq u_{k+1} \\ 0 & \text{en otro caso} \end{cases}$$

$$B_i^n(t) = \frac{t - u_i}{u_{i+n} - u_i} B_i^{n-1}(t) + \frac{u_{i+n+1} - t}{u_{i+n+1} - u_{i+1}} B_{i+1}^{n-1}(t)$$

⚠ Además de los pts de control, hay que definir el vector de knots u_i (ts donde cambian las funciones "activas")

- Combinaciones afines y convexas
- **Control local**
- **Continuidad C^{n-1}** (ahorrando puntos de control!)
- No interpolantes*

Las B-Splines son otro tipo de spline, con una base diferente. En el fondo son equivalente a una spline de Bézier, pero se definen de otra manera, que utiliza menos puntos de control y garantiza mayor suavidad (más derivadas paramétricamente continuas).

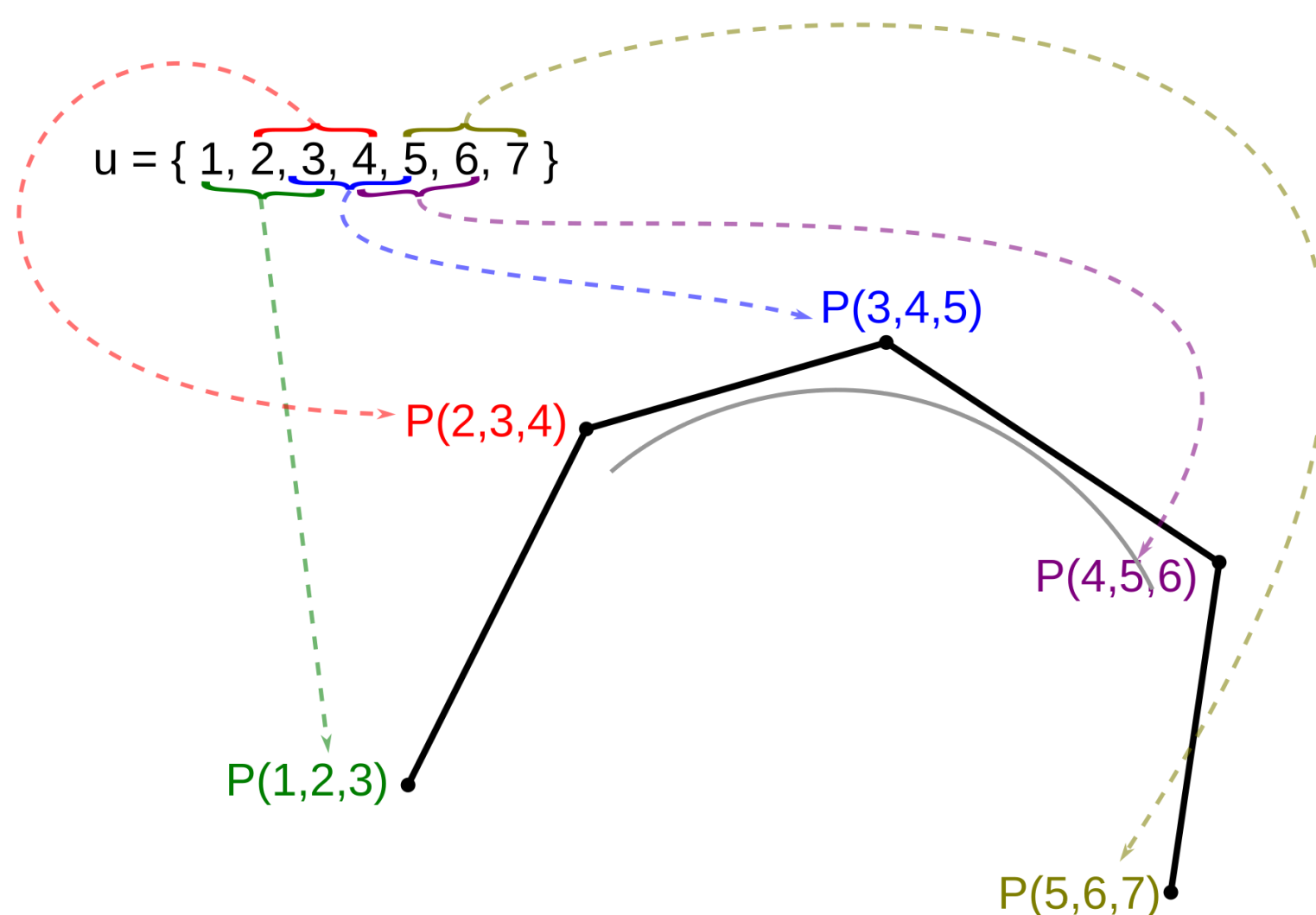
Las funciones bases se definen recursivamente, pero el resultado no deja de ser pesos que forman cada punto de la curva mediante una combinación afín y convexa, así que mantendrá muchas de las propiedades de Bézier.

Mirando el paso base de la recursión (B_k^0) se puede observar que tienen soporte compacto (que solo valen algo diferente de cero en un tramo). Esto se traducirá en control local (donde una función base vale 0, su punto de control no influye).

Sin embargo, lo que más debería llamar la atención es la presencia de unos valores extraños u_i . Para definir una de estas curvas, además de definir las posiciones de los puntos de control y el grado de la misma (el grado ya no depende de la cantidad de pts de control), hay que definir además un "vector de knots". Esto es, un vector de valores reales (tamaño: cant de pts de control + grado - 1) no decrecientes. Estos valores determinarán de dónde hasta dónde va el parámetro de la curva, y en qué rangos de ts se "activa" cada función base.

Al igual que pasó con Bézier, no es necesario recordar estas fórmulas; a continuación veremos la generalización del algoritmo de De Casteljau, y esto nos permitirá evaluar la curva y entender mejor sus propiedades y el rol de los knots.

B-SPLINES: BLOOSOMING



Se asocian n (grado) knots a cada pto de control.
(no importa el orden, $P(1,2,3)=P(1,3,2)=P(3,1,2)=\dots$)

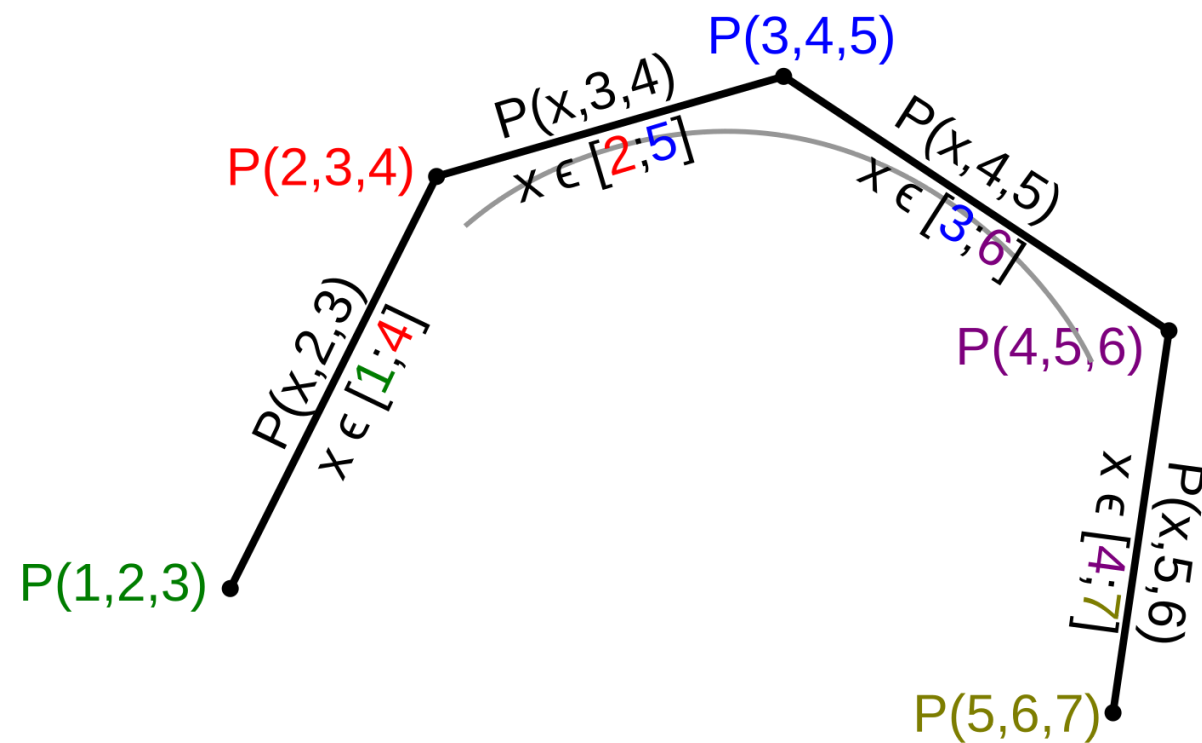
Si la curva es de grado n , se deben ir tomando del vector de knots n -uplas (superpuestas, como se muestra en la figura) de valores para asociar a los puntos de control. En lugar de "numerar" a los pts de control, los identificaremos por sus knots asociados. Y se aplicarán las siguientes reglas (se toma como ejemplo una B-Spline de grado 3, por eso hay 3 knots para cada P):

- No importa el orden de los knots de un punto. Es decir, que $P(1,2,3)$ es lo mismo que $P(1,3,2)$ o $P(3,1,2)$ o cualquier otra permutación de los mismos 3 valores.
- El punto $P(t,t,t)$ (donde los 3 valores son iguales) es el punto de la curva para el valor del parámetro t (o sea, $F(t)$).
- Si dos puntos difieren en un solo knot, se puede interpolar entre ellos: $P(t,2,3) = (1-a)*P(1,2,3) + a*P(4,2,3)$, con $a = (t-1)/(4-1)$ (es decir, como si t barriera el segmento al variar de 1 a 4, los valores del knot variable, en lugar de 0 a 1 como pasaba en Bézier).

B-SPLINES: BLOOSOMING

Sobre cada segmento se puede interpolar (afín):

$$P(a, (1-u) \mathbf{b} + u \mathbf{c}, d) = (1-u) P(a, \mathbf{b}, d) + u P(a, \mathbf{c}, d)$$



Ejemplo: combinando $P(1,2,3)$ y $P(4,2,3)$ se puede obtener $P(t,2,3)$, con t entre 1 y 4. El punto medio del segmento será $P(2.5,2,3)$, ya que 2.5 está a mitad camino entre 1 y 4.

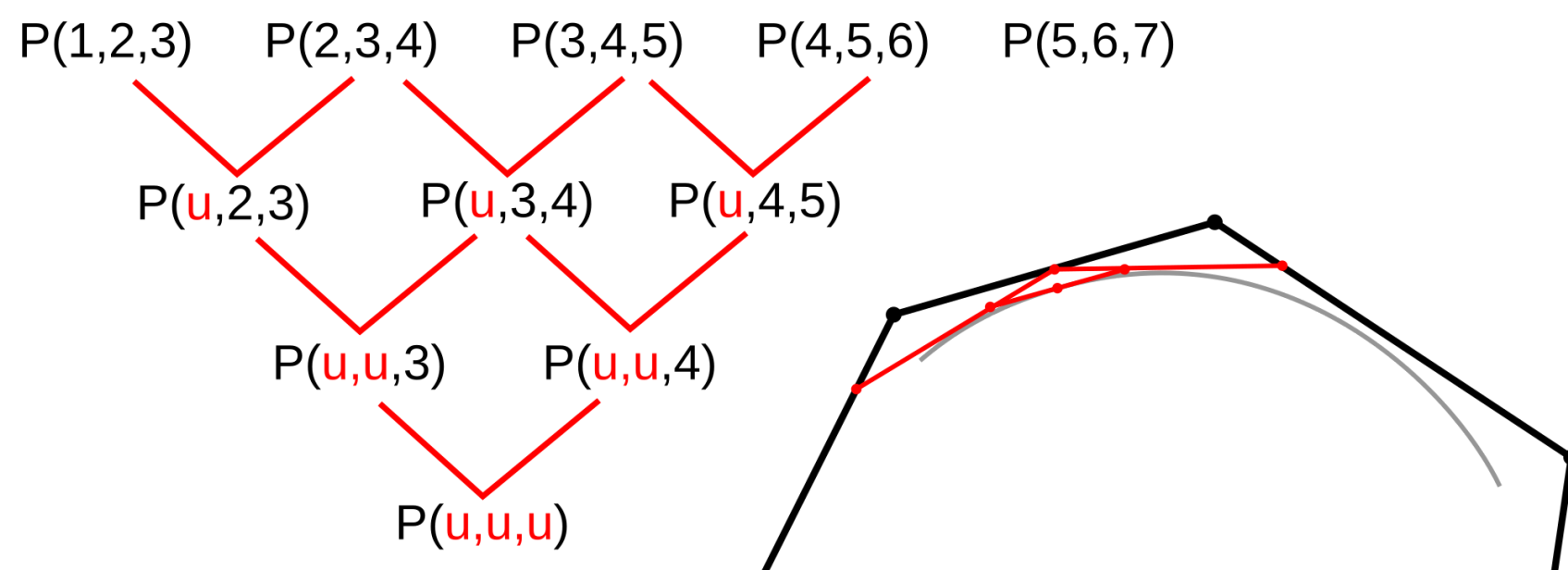
Por esto a veces nos referiremos a cada segmento como el segmento A-B siendo A y B los valores entre los que se interpola (El primer segmento sería el 1-4, el segundo sería el 2-5, etc).

Diremos que un segmento "contiene" al parámetro t si t está entre su rango A-B. Por ej, los 3 primeros segmentos contienen a $t=3.5$, pero el último no (el segmento 4-7). Para obtener el punto de la curva $f(3.5)$ el último segmento no será necesario (es decir, que el peso del último punto será 0), y aquí se observa nuevamente la idea del control local.

Sin embargo, notar que si la curva es de grado 3, necesitamos 3 segmentos (4 puntos) para poder hacer la cantidad suficiente de pasos (recordar De Casteljau) para llegar a un único punto que será el punto de la curva. Si queremos evaluar la curva, por ejemplo en $t=2.5$, no podremos hacerlo porque solo hay 2 segmentos que lo contienen. Es por esto que los valores "válidos" para t van desde el mayor knot del primer pto de control (3 en este ejemplo) hasta el menor knot del último pto de control (5 en este ejemplo).

B-SPLINES: BLOOSOMING

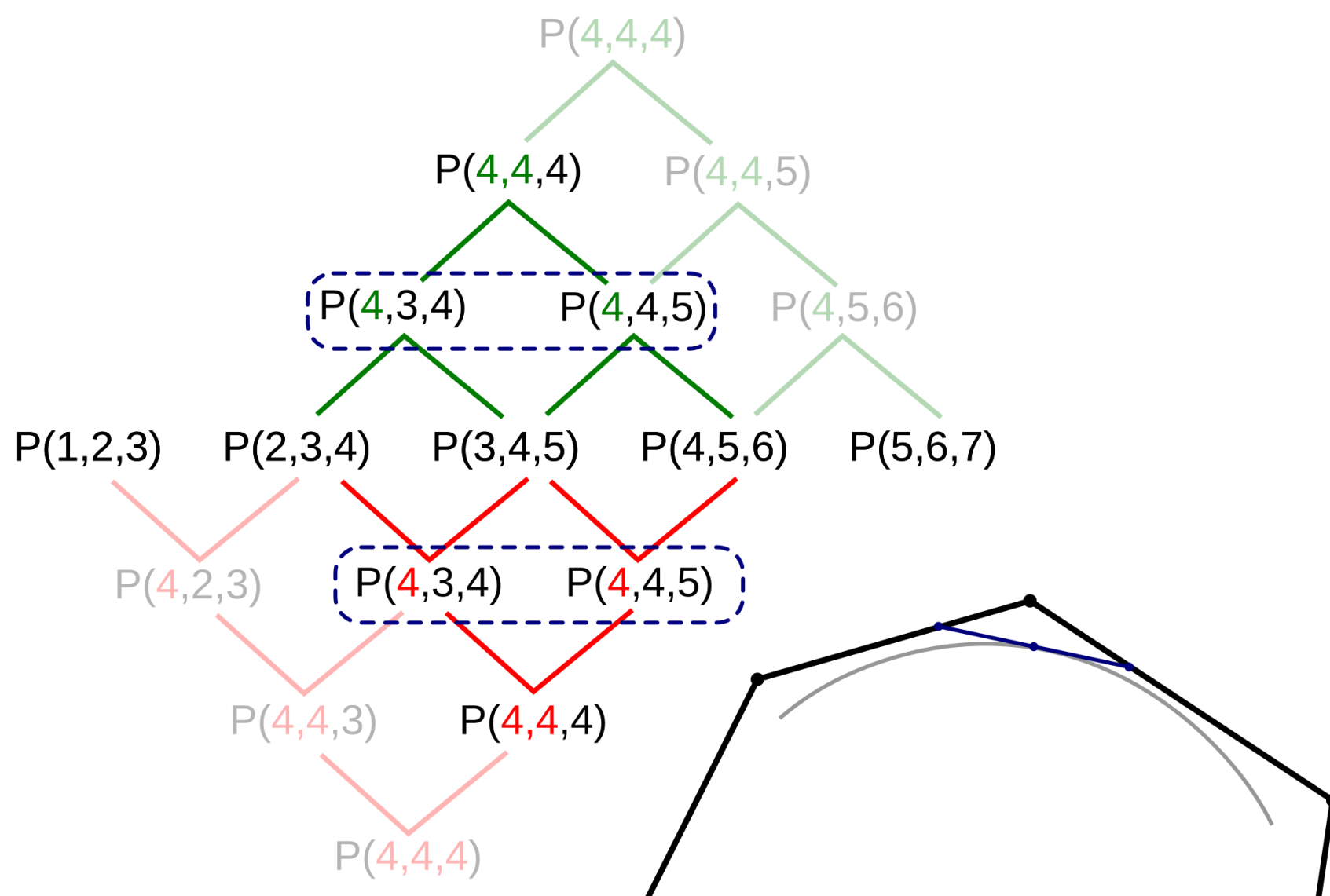
Para obtener $F(u)$, se interpola hasta llegar a $P(u,u,u)$.
Solo se usan los segmentos que contienen a u .



Aquí se esquematiza la idea de la nota anterior. En una B-Spline de grado N , solo N segmentos del polígono de control "contendrán" al parámetro. En cada uno se interpola, generando un nuevo sub-polígono de $N-1$ lados (como pasaba en De Casteljau, solo que los t s no van de 0 a 1 en cada lado), hasta llegar a un único punto. En cada interpolación, t aparece entre los knots del punto resultando reemplazando al valor que difería entre los extremos del segmento. Luego de N pasos, llegamos a $P(t,t,t)$ (todos los knots valen t), y ese es el punto que buscamos, que forma parte de la curva.

En la figura se muestra el "árbol" de interpolaciones correspondiente a un valor del parámetro entre 3 y 4.

B-SPLINES: BLOOSOMING



Aquí se muestra el "árbol" de cálculos para cuando t está entre 3 y 4 (en rojo), y para cuando está entre 4 y 5 (el verde).

Pero si queremos evaluar justo $t=4$, podríamos hacerlo con cualquiera de los dos. Y en ambos veremos que al hacer las combinaciones afines hay un pto de control que no influye (cuyo peso será 0, $P(1,2,3)$ para al árbol rojo, o $P(5,6,7)$ para el verde). Más importante que eso es notar que llegamos al punto que queremos ($P(4,4,4)$) una iteración antes; esto es porque como el knot ya "estaba" desde el comienzo asociado a un pto, requerimos de una interpolación menos para hacerlo aparecer en los otros $N-1$ knots.

Aquí se verifica la continuidad. $P(4,4,4)$ tendrá el mismo valor si se calcula por el "lado" rojo, o por el verde. Si pensamos como pasaba con De Casteljau que los dos puntos de la iteración anterior tienen que ver con la derivada (lo recuadrado en azul), entonces vemos que las derivadas también coinciden si se calcula por el "lado" verde o rojo. Y si pensamos en el nivel anterior como una representación de la 2da derivada, vemos que también coincide. Sobre la 3er derivada ya no hay información, y podría no ser nula, así que podría ser discontinua (en general lo será).

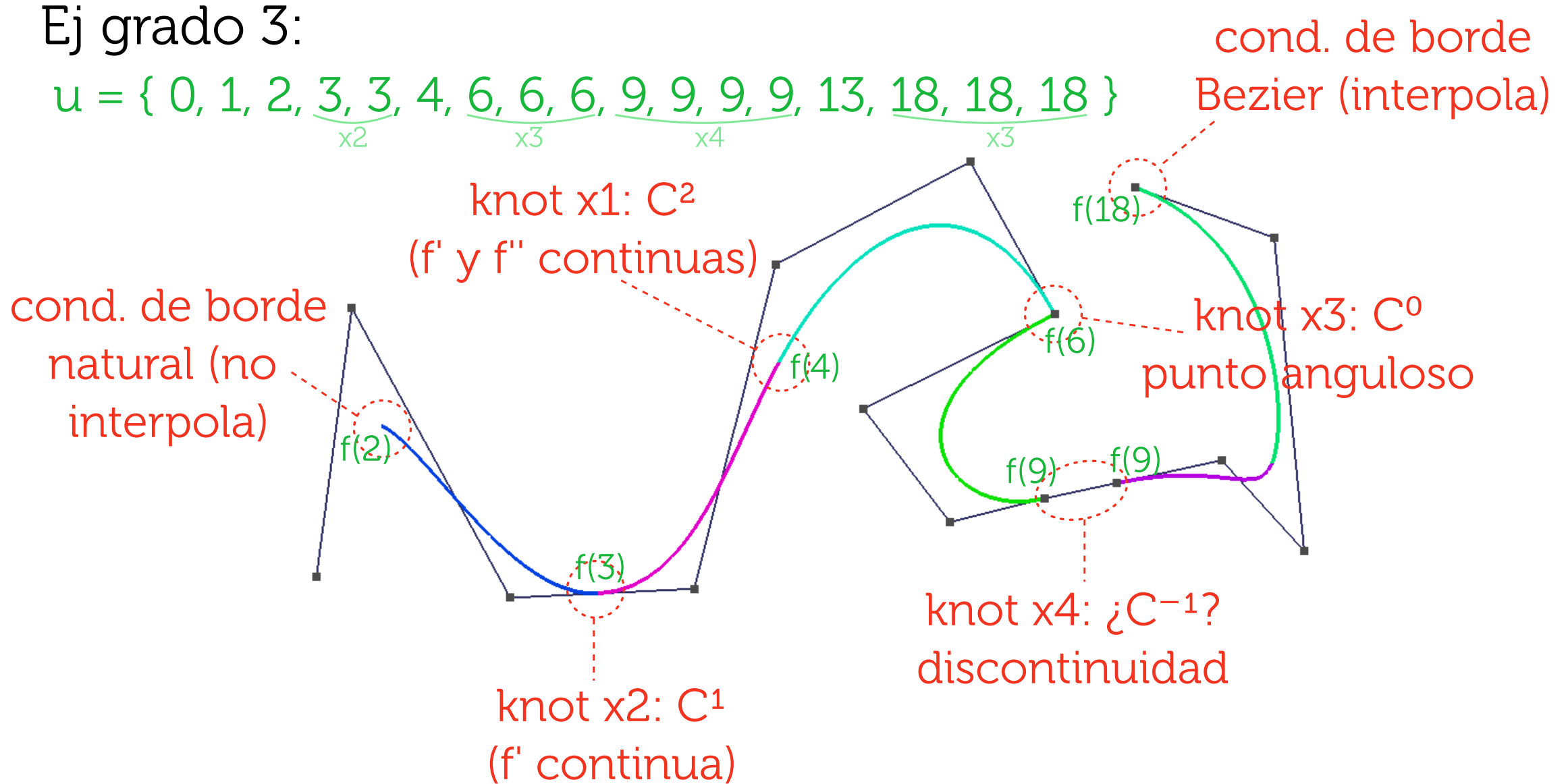
Por esto decimos que las BSplines nos dan continuidad C^{N-1} . Sera infinita dentro de cada "tramo" (entre knots) por la infinita derivabilidad de los polinomios, pero solo $N-1$ en los puntos donde se unen los tramos.

En conclusión tenemos control local y mejor continuidad que las splines de Bézier simples, y con menos esfuerzo y menos puntos de control.

BSPLINES: EJEMPLO Y GENERALIDAD

Ej grado 3:

$$u = \{ 0, 1, 2, \underbrace{3, 3}_{x_2}, 4, \underbrace{6, 6, 6}_{x_3}, \underbrace{9, 9, 9, 9}_{x_4}, 13, \underbrace{18, 18, 18}_{x_3} \}$$



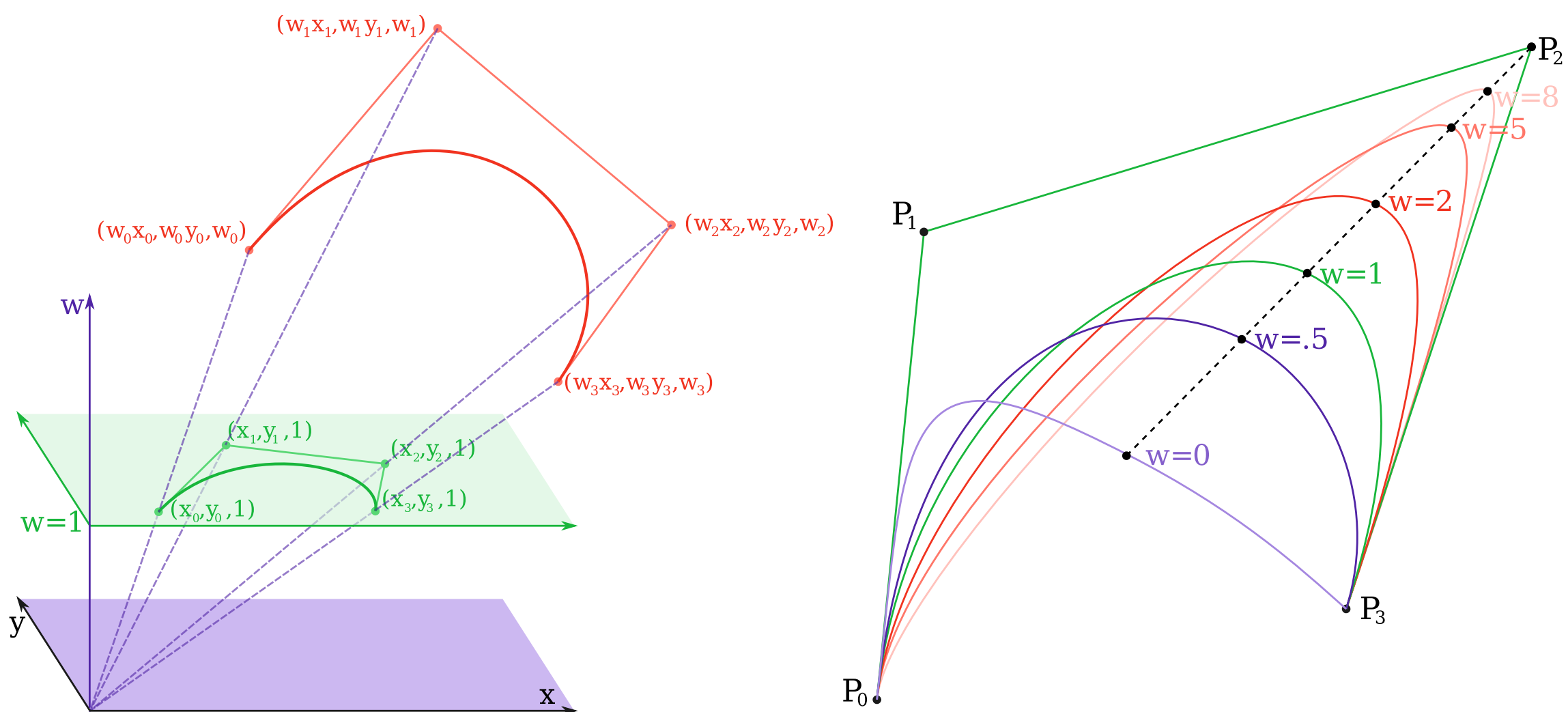
Pero hay más... se puede decir que cada vez que un knot se repite en el vector de knots iniciales (debía ser no-decreciente, pero puede haber repetidos) se reducen los pasos de interpolación necesarios para hallar ese punto, y con ello también se pierde un grado más de continuidad. En una curva de grado 3 como el ejemplo, si un knot se repite 3 veces, la curva interpolará a ese pto de control, pero solo tendrá continuidad C^0 ahí. Más aún, si se repite 4 veces la curva es discontinua! (se puede hacer cosas raras con el vector de knots).

Dado que la definición del vector de knots no es algo intuitivo para un usuario final, suele ser algo que gestiona internamente el programa. Como punto de partida se suele tomar un vector de knots equiespaciado, o con "condiciones de borde Bézier" (repetiendo N veces primer y último valor para que la curva interpole los extremos).

Lo interesante es que con esta herramienta podemos representar en un solo tipo de curva todo lo que vimos hasta ahora (y una Bézier, o una spline de Bézier no es más que un caso particular de BSpline), controlar a gusto el grado de continuidad, elegir cuáles pts interpolar y cuáles no, etc.

Finalmente, el vector de knots, además de alterar la continuidad, nos permite modificar la "velocidad" de la curva en cada tramo. Notar por ejemplo que entre $f(3)$ y $f(4)$ hay una distancia (en long de arco) muy similar a la que se ve entre $f(9)$ y $f(18)$. Sin embargo si recorremos la curva avanzando 1 en t por cada segundo, pasar de $f(3)$ a $f(4)$ tomaría 1 segundo, pero de $f(9)$ a $f(18)$ tomaría 9 segundos. Si usamos la curva para animar algo, variar las diferencias entre knots consecutivos se traduce en variar la velocidad con que se recorre la curva; entonces además de controlar la suavidad, los knots nos permiten controlar los cambios de velocidad de la parametrización.

CURVAS RACIONALES



✓ Al definir los puntos de control como $\{wx, wy, w\}$, w altera la "influencia" del punto sobre la curva

Las curvas racionales son curvas con puntos de control definidos en coordenadas homogéneas. Es decir, con la coordenada adicional w . Pero en este caso w no será una coordenada independiente, sino será un factor que escalará por completo al punto original.

Si un punto de control era $P=\{x,y,z\}$, al pasar al espacio homogéneo comienza siendo $P=\{x,y,z,1\}$. Si permitimos variar w independientemente de las demás coordenadas, ese cambio afecta a la "proyección" del pto de control sobre el plano $w=1$ (que es el que nos interesa).

Pero si pensamos al punto como $\{wx, wy, wz, w\}$, entonces al variar w (y escalar entonces x,y,z de la misma forma), nos mantenemos sobre la misma "recta" del espacio proyectivo (recordar que los puntos/elementos de este espacio eran rectas por el origen). En este caso entonces la proyección del pto de control no cambia (en el espacio "común", al dividir por w , se ve igual), pero la diferencia es que el punto funciona distinto para las interpolaciones con las que se construye la curva (recordar la diferencia entre lineal e hiperbólica). El efecto es que w termina funcionando como un factor que altera el "peso" del punto, que permite hacer que el punto atraiga más o menos a la curva.

La figura de la derecha muestra varias curvas en $\mathbb{R}^2$ definidas con los mismos pts de control en x,y , pero variando w para P_2 . Notar que cuando w aumenta, la influencia de P_2 aumenta también, la curva se aproxima más a ese punto. Por el contrario, cuando w disminuye la influencia del punto también (hasta puede anularse cuando $w=0$, o "repeler" a la curva cuando $w<0$).

Como ventaja adicional nos permite representar curvas que no sería posible de otra forma, como un círculo (ningún polinomio finito podría representarlo exactamente).

CURVAS RACIONALES: EJEMPLO

$$P = \underbrace{\{wx, wy, wz\}}_{wX}, w = \{wX, w\} \Rightarrow P^* = \frac{wX}{w}$$

$$\begin{aligned} \underbrace{\{wX_{01}, w_{01}\}}_{P_{01}} &= (1-u) \underbrace{\{w_0 X_0, w_0\}}_{P_0} + u \underbrace{\{w_1 X_1, w_1\}}_{P_1} = \\ &= \left\{ \underbrace{(1-u)w_0 X_0 + u w_1 X_1}_{w_{01} X_{01}}, \underbrace{(1-u)w_0 + u w_1}_{w_{01}} \right\} \end{aligned}$$

$$P_{01}^* = \frac{(1-u)w_0 X_0 + u w_1 X_1}{w_{01}} = \frac{(1-u)w_0}{w_{01}} X_0 + \frac{u w_1}{w_{01}} X_1$$

✓ Operar con w como si fuera una coordenada cualquiera más, y solo para "dibujar" hacer la división por el w resultante

Para calcular un pto de una curva racional, el procedimiento comienza igual que para una no racional, tratando a la coordenada w como una más. Por ejemplo si estamos en P^2 , podemos tratar a w como si fuera z en R^3 . Solo cuando toque "dibujar" el punto hacemos la división por el w resultante para ver dónde se "proyectaría". Pero durante la marcha de cálculos de las interpolaciones afines de De Casteljaou o Bloosoming, operamos sobre w igual que sobre x , y o z (es decir, hacemos las interpolaciones lineales en el espacio homogéneo, y al final proyectamos el resultado en $w=1$, pero no los puntos intermedios).

Lo importante a tener en cuenta, es plantear el punto como $\{wx, wy, w\}$ (no como $\{x, y, w\}$) para que w tenga efectivamente el sentido que queremos.

CURVAS NURBS

- NU = Non-Uniform
 - vector de knots arbitrario
- R = Rational
 - coordenadas homogéneas
 - w alterando el "peso" del pto de control
- BS = B-Splines
 - base: Cox/de Boor

Si juntamos todo (usar BSpline como base, poder jugar con el vector de knots, definir los pts de control con coordenadas homogéneas) obtenemos las curvas NURBS. Es por esto que se prefieren frente a Bézier o BSplines no racionales en muchos CADs o sistemas similares (donde importa la precisión), principalmente para trabajar en 3D (ya sea curvas en 3D, o como veremos en la próxima clase, la extensión de estas idea a superficies).

¿QUÉ HAY QUE SABER?

- Definiciones generales
 - Curvas paramétricas/Parametrización
 - Derivada y Continuidad (Geométrica vs. Paramétrica)
- **Curvas de Bézier**  *con mucho detalle*
 - **Definición y propiedades**
 - **Algoritmo de De Casteljau**
 - Splines de Bézier
- BSplines  *detalle medio*
- Curvas Racionales y NURBS  *solo concepto general*